

Symbiosis Institute of Technology, Nagpur

DEPARTMENT OF COMPUTER SCIENCE • JAVASCRIPT PROGRAMMING LAB

Student: Tanmay Sankulwar
PRN: 24070521058
Course: Semester V (B.Tech)

Experiment No. 4

Student Name: Tanmay Sankulwar

PRN: 24070521058

File Path: EXP4/index.html , EXP4/script.js , EXP4/style.css

Experiment Title

SIT NAGPUR | Vehicle Validator (Vehicle Registration Validator)

Software / Tools Required

- 1. Visual Studio Code / Antigravity IDE
- 2. Google Chrome (or modern web browser)
- 3. HTML5
- 4. CSS3
- 5. JavaScript (ES6)

Experiment Program Code

Task 4.a — Vehicle Registration Input and Dynamic Details Panel

File: EXP4/index.html

The HTML file implements a vehicle verification portal adhering to Indian standard vehicle registration patterns (e.g., MH12AB1234). It includes real-time validation rule indicators, an action trigger button, a feedback banner, and a parsed details grid.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>SIT NAGPUR | Vehicle Validator</title>
  <link href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <div class="app-container">
    <header class="app-header">
      <h1>Vehicle Validator Portal</h1>
      <p class="subtitle">JavaScript Experiment - 4</p>
    </header>
```

```

<main class="main-content">
  <div class="card">
    <div class="card-header">
      <h2>Vehicle Registration Validator</h2>
      <p class="card-desc">Verify Indian vehicle registration formats against official schema rules.
    </p>
    </div>

    <div class="form-group">
      <label for="vehicleInput">Vehicle Registration Number</label>
      <input type="text" id="vehicleInput" placeholder="e.g., MH12AB1234" maxlength="12">
      <span class="helper-text">Enter 10-character code without spaces or hyphens</span>
    </div>

    <ul class="rules-list">
      <li class="rule-item invalid" id="ruleNotEmpty">
        <span class="indicator"></span> Registration number is not empty
      </li>
      <li class="rule-item invalid" id="ruleLength10">
        <span class="indicator"></span> Length is exactly 10 characters
      </li>
      <li class="rule-item invalid" id="ruleState">
        <span class="indicator"></span> First 2 characters are uppercase letters (State Code)
      </li>
      <li class="rule-item invalid" id="ruleDistrict">
        <span class="indicator"></span> Next 2 characters are digits (District Code)
      </li>
      <li class="rule-item invalid" id="ruleSeries">
        <span class="indicator"></span> Next 2 characters are uppercase letters (Series)
      </li>
      <li class="rule-item invalid" id="ruleNumber">
        <span class="indicator"></span> Last 4 characters are digits (Vehicle Number)
      </li>
    </ul>

    <button class="action-btn" onclick="validateVehicleForm()">Validate Registration</button>

    <div id="vehicleFeedback" class="feedback-banner"></div>

    <div id="plateDetails" class="details-panel" style="display: none;">
      <h3>Parsed Registration Details</h3>
      <div class="details-grid">
        <div class="detail-tile">
          <span class="tile-label">State Code</span>
          <span class="tile-val" id="detailState"></span>
        </div>
        <div class="detail-tile">
          <span class="tile-label">District Code</span>
          <span class="tile-val" id="detailDistrict"></span>
        </div>
        <div class="detail-tile">
          <span class="tile-label">Series Code</span>
          <span class="tile-val" id="detailSeries"></span>
        </div>
        <div class="detail-tile">
          <span class="tile-label">Vehicle ID</span>
          <span class="tile-val" id="detailId"></span>
        </div>
      </div>
    </div>
  </div>
</main>

<footer class="app-footer-brand">
  <p><b>Developed By</b></p>
  <p class="developer-name">Tanmay Sankulwar</p>
  <p>PRN : <b>24070521058</b></p>
</footer>
</div>

<script src="script.js"></script>

```

```
</body>
</html>
```

Task 4.b — Component Styling and Validation States

File: EXP4/style.css

Key CSS styling rules that manage visual hierarchy, valid/invalid state indicator badges, glassmorphic cards, and parsed detail tiles:

```
:root {
  --primary: #2563eb;
  --primary-dark: #1d4ed8;
  --success: #10b981;
  --error: #ef4444;
  --bg-dark: #0f172a;
  --text-main: #1e293b;
  --text-muted: #64748b;
  --border-color: #cbd5e1;
}

.rule-item {
  display: flex;
  align-items: center;
  gap: 10px;
  font-size: 14px;
  color: var(--text-muted);
  margin-bottom: 8px;
  transition: color 0.3s ease;
}

.rule-item .indicator {
  width: 14px;
  height: 14px;
  border-radius: 50%;
  background-color: var(--border-color);
  display: inline-block;
  transition: background-color 0.3s ease;
}

.rule-item.valid {
  color: var(--success);
  font-weight: 500;
}

.rule-item.valid .indicator {
  background-color: var(--success);
}

.rule-item.invalid-touched {
  color: var(--error);
}

.rule-item.invalid-touched .indicator {
  background-color: var(--error);
}

.details-panel {
  margin-top: 25px;
  background: #f8fafc;
  border-radius: 12px;
  padding: 20px;
  border: 1px solid #e2e8f0;
}

.details-grid {
  display: grid;
}
```

```

grid-template-columns: repeat(2, 1fr);
gap: 15px;
margin-top: 12px;
}

```

Task 4.c — Real-Time Input Sanitization and Schema Validation

File: EXP4/script.js

The JavaScript code sanitizes user input by stripping whitespace and converting to uppercase (`replace(/\s+/g, '').toUpperCase()`), checks all 6 schema criteria using substrings and RegExp tests, displays tailored feedback alerts, and decomposes the string into State, District, Series, and Vehicle Number.

```

document.addEventListener('DOMContentLoaded', function() {
  const vehicleInput = document.getElementById('vehicleInput');

  // Real-time listener for vehicle registration number
  vehicleInput.addEventListener('input', function() {
    // Strip spaces and normalize to uppercase for the input
    let val = vehicleInput.value.replace(/\s+/g, '').toUpperCase();
    vehicleInput.value = val;

    updateVehicleRules(val, false);
  });

  // Real-time vehicle rule verification
  function updateVehicleRules(val, forceErrorState = false) {
    // 1. Not Empty Check
    const isEmpty = val.length > 0;
    setRuleState('ruleNotEmpty', isEmpty ? 'valid' : (forceErrorState ? 'invalid-touched' : 'invalid'));

    // 2. Length Check
    const isLength10 = val.length === 10;
    let lengthState = 'invalid';
    if (isLength10) {
      lengthState = 'valid';
    } else if (val.length > 10 || (forceErrorState && !isLength10)) {
      lengthState = 'invalid-touched';
    }
    setRuleState('ruleLength10', lengthState);

    // 3. First two characters: Uppercase alphabets (State Code)
    let stateStatus = 'invalid';
    if (val.length >= 2) {
      const isStateValid = /^[A-Z]{2}$/.test(val.substring(0, 2));
      stateStatus = isStateValid ? 'valid' : 'invalid-touched';
    } else if (forceErrorState) {
      stateStatus = 'invalid-touched';
    }
    setRuleState('ruleState', stateStatus);

    // 4. Next two characters: Digits (District Code)
    let districtStatus = 'invalid';
    if (val.length >= 4) {
      const isDistrictValid = /^[0-9]{2}$/.test(val.substring(2, 4));
      districtStatus = isDistrictValid ? 'valid' : 'invalid-touched';
    } else if (forceErrorState) {
      districtStatus = 'invalid-touched';
    }
    setRuleState('ruleDistrict', districtStatus);

    // 5. Next two characters: Uppercase alphabets (Series)
    let seriesStatus = 'invalid';
    if (val.length >= 6) {
      const isSeriesValid = /^[A-Z]{2}$/.test(val.substring(4, 6));
      seriesStatus = isSeriesValid ? 'valid' : 'invalid-touched';
    }
  }
}

```

```

    } else if (forceErrorState) {
        seriesStatus = 'invalid-touched';
    }
    setRuleState('ruleSeries', seriesStatus);

    // 6. Last four characters: Digits (Vehicle Number)
    let numberStatus = 'invalid';
    if (val.length >= 10) {
        const isNumberValid = /^[0-9]{4}$/.test(val.substring(6, 10));
        numberStatus = isNumberValid ? 'valid' : 'invalid-touched';
    } else if (forceErrorState) {
        numberStatus = 'invalid-touched';
    }
    setRuleState('ruleNumber', numberStatus);

    return isEmpty && isLength10 &&
        /^[A-Z]{2}$/.test(val.substring(0, 2)) &&
        /^[0-9]{2}$/.test(val.substring(2, 4)) &&
        /^[A-Z]{2}$/.test(val.substring(4, 6)) &&
        /^[0-9]{4}$/.test(val.substring(6, 10));
}

function setRuleState(elementId, stateClass) {
    const el = document.getElementById(elementId);
    if (!el) return;
    el.className = `rule-item ${stateClass}`;
}

// Vehicle Validation Submit
window.validateVehicleForm = function() {
    const val = document.getElementById('vehicleInput').value.trim();
    const plateDetailsPanel = document.getElementById('plateDetails');

    const isValid = updateVehicleRules(val, true);

    if (isValid) {
        showFeedback('vehicleFeedback', `Verification Successful! <strong>${val}</strong> is a valid registration number.`, 'success');

        document.getElementById('detailState').textContent = val.substring(0, 2);
        document.getElementById('detailDistrict').textContent = val.substring(2, 4);
        document.getElementById('detailSeries').textContent = val.substring(4, 6);
        document.getElementById('detailId').textContent = val.substring(6, 10);

        plateDetailsPanel.style.display = 'block';
    } else {
        showFeedback('vehicleFeedback', 'Verification Failed! Please correct the highlighted formatting errors.', 'error');
        plateDetailsPanel.style.display = 'none';
    }
};

function showFeedback(id, message, type) {
    const banner = document.getElementById(id);
    if (!banner) return;
    banner.innerHTML = message;
    banner.className = `feedback-banner ${type}`;
    banner.style.display = 'block';
}

```

Output

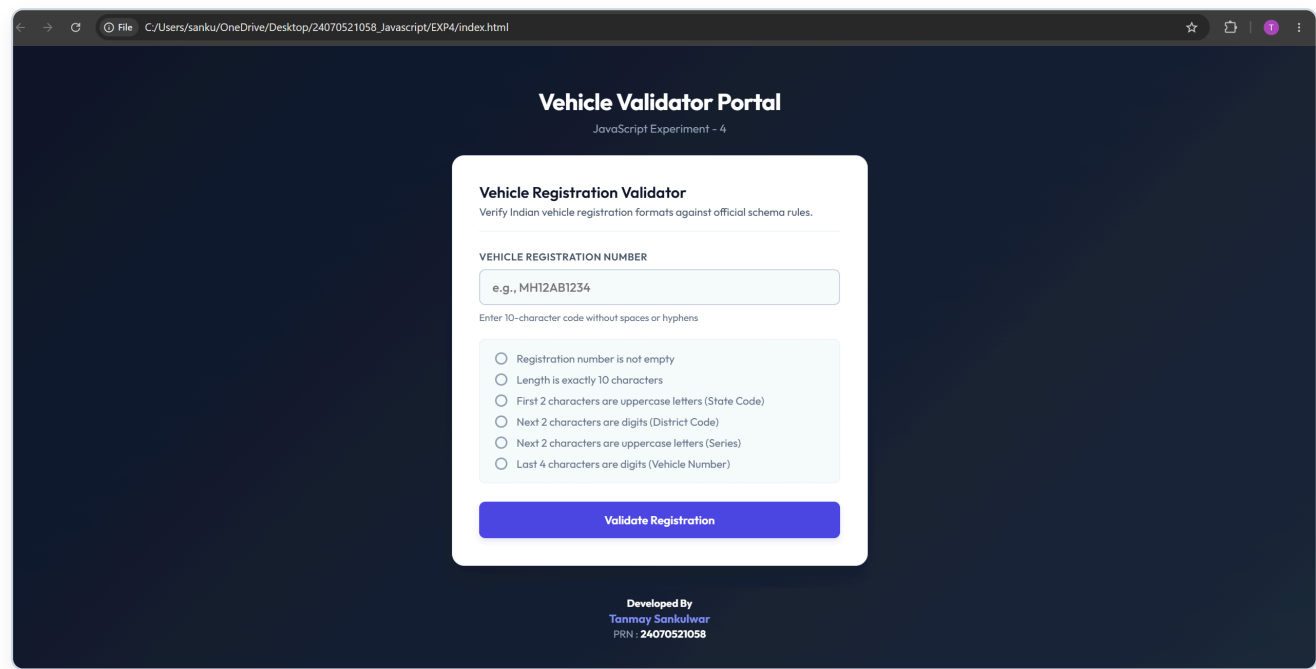
- Live Validation Feedback:** As registration digits are typed (e.g., `MH12AB1234`), each rule turns green in real time once matched.
- Schema Verification:** When clicking **Validate Registration**, invalid or incomplete entries display an alert banner:

"Verification Failed! Please correct the highlighted formatting errors."

3. **Parsed Detail Grid:** When verified valid, a green success banner appears and the `plateDetails` grid reveals: - **State Code:** `MH` - **District Code:** `12` - **Series Code:** `AB` - **Vehicle ID:** `1234`

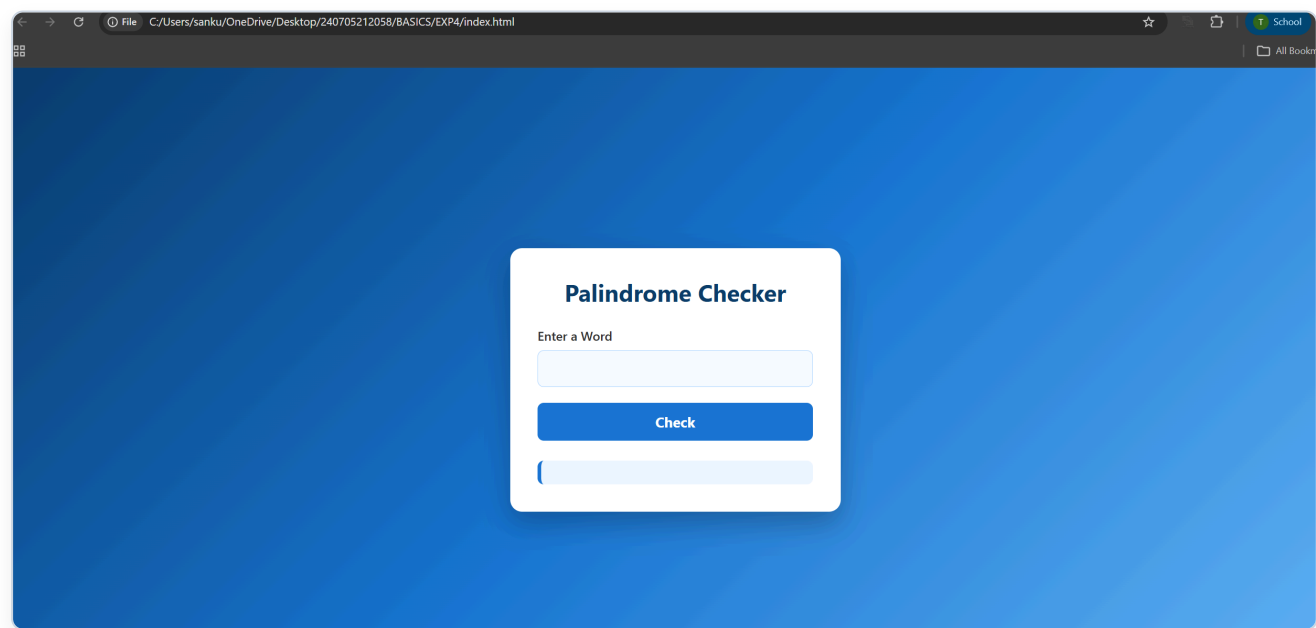
Screenshot

Schema Checklist & Real-Time Input State



Vehicle Validator Interface

Parsed Vehicle Registration Details



Vehicle Registration Output

Result / Conclusion

Experiment 4 demonstrated string sanitization, substring extraction, and Regular Expression matching in JavaScript. The implementation successfully enforced the 10-character Indian motor vehicle registration standard, gave instant visual cues to the user, and extracted structural registration segments dynamically.